

1 Conventions and Definitions

1.1 Definitions

Definition 1 A scalar, M_0 is defined using a type t_0

Definition 2 A vector or 1D matrix, M_1 , is defined using a type t_1 and positive integer n_1 , representing the dimensions/shape.

- $M_1[j]$ is a scalar of type $\mathbb{T}$ of size s_1
- M_1 is stored as a linear array at address m_1 such that
 1. Address of $M_1[j] = m_1 + s_1 \times j$

Definition 3 A 2D matrix, M_2 , is defined using a type t_2 and positive integers n_X, n_Y , representing the dimensions/shape.

- $M_2[i]$ is a 1D matrix of length n_Y and type t_2
- $M_2[i, j]$ is a scalar of type t_2 and size s_2
- M_2 is stored as a linear array at address m_2 such that
 1. Address of $M_2[i] = m_2 + s_2 \times (i \times n_Y)$
 2. Address of $M_2[i, j] = m_2 + s_2 \times (i \times n_Y + j)$

Definition 4 A 3D matrix, $\mathbf{M}$, is defined using a type $\mathbb{T}$ and positive integers n_X, n_Y, n_Z , representing the dimensions/shape.

- $\mathbf{M}[x]$ is a 2D matrix of dimensions n_Y, n_Z and type $\mathbb{T}$
- $\mathbf{M}[x, y]$ is a vector of length n_Z and type $\mathbb{T}$
- $\mathbf{M}[x, y, z]$ is a scalar of type $\mathbb{T}$
- $\mathbf{M}$ is stored as a linear array at address $m_{\mathbf{M}}$ such that
 1. Address of $\mathbf{M}[x] = m_{\mathbf{M}} + x \times (n_Y \times n_Z)$
 2. Address of $\mathbf{M}[x, y] = m_{\mathbf{M}} + (x \times n_Y \times n_Z) + (y \times n_Z)$
 3. Address of $\mathbf{M}[x, y, z] = m_{\mathbf{M}} + (x \times n_X \times n_Y) + (y \times n_Z) + z$

Write a C function, called `dotprod`, that takes as input

1. x , type is (M_1, n, float)
2. y , type is (M_1, n, float)
3. n , type is (M_0, int)

and returns

1. s , type is (M_0, float)

such that

1. $s \leftarrow \sum_i x_i \times y_i$
1. Produce highly optimized C code.
2. Use AVX2 intrinsics wherever possible.
3. Do **not** assume that pointers, x, y , are aligned on 64-byte boundaries
4. Assume that there is no overlap in the memory accessed using x, y